

CATALOGGUARD LITE

상품 카탈로그 데이터 품질 검수 서비스

프로젝트 한 줄 소개

서로 다른 합성 공급사 CSV 를 JSON 프로필로 표준화하고, 상품 오류 검수와 PostgreSQL staging 적재·조회까지 연결한 Python·FastAPI 백엔드 프로젝트입니다.

프로젝트 요약

항목	내용
형태	개인 프로젝트 · 취업 포트폴리오용 MVP 고도화
담당	검수 규칙, FastAPI API, PostgreSQL 저장·조회, Redis·Celery, ETL 프로필 2 종, 테스트·CI, 배포 검증
핵심 기술	Python, FastAPI, PostgreSQL, SQLAlchemy, Alembic, Redis, Celery, pandas, Streamlit
운영·검증	Docker Compose, GitHub Actions, Railway, AWS EC2·RDS staging

검증된 현재 상태

894 passed 전체 PostgreSQL 회귀	Test #35 GitHub Actions 성공	PostgreSQL 18.4 ETL staging·조회 검증	Version 5 sale_price 규칙 반영
---------------------------------------	--------------------------------------	---	--------------------------------------

핵심 성과

검수 실행·저장, 이력 검색·상세 조회, Redis·Celery 비동기 검수 API 를 구현하였습니다.
공급사 JSON 프로필 2 종을 공통 ETL 코드로 처리하고 정상·reject·summary 산출물을 생성하였습니다.
표준 상품을 PostgreSQL staging 에 배치 적재하고, 무결성·중복·rollback·DB 제약조건을 검증하였습니다.
ETL 적재 배치·상품 조회 API 에 검색·SQL 페이지네이션·404·배치 격리를 적용하였습니다.

GitHub [psy0635-ctrl/catalogguard-lite](#) Demo [CatalogGuard Lite Streamlit 앱](#)

1. 문제 정의와 사용자 가치

기능 나열보다 상품 데이터가 왜 검수·표준화되어야 하는지와 사용자가 얻는 결과를 먼저 정의했습니다.

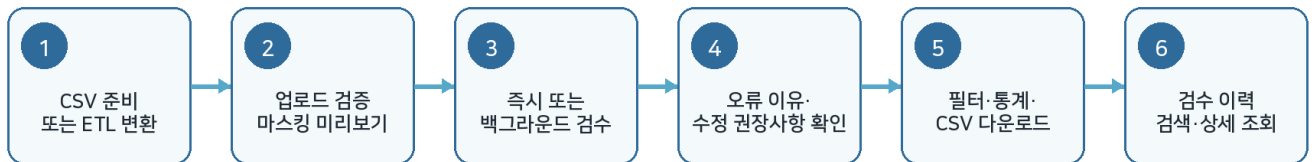
문제 정의

쇼핑몰 상품 카탈로그는 공급사와 운영 담당자마다 컬럼명과 입력 방식이 달라 같은 의미가 여러 형태로 저장될 수 있습니다. 오류가 누적되면 검색·필터·재고 관리·상품 노출의 정확도가 낮아지고, 사람이 CSV 를 반복 확인해야 하는 비용이 커집니다.

반복되는 문제	예시	서비스의 대응
형식 불일치	price 에 문자, 필수값 누락	업로드 검증과 규칙 기반 오류 탐지
표기 불일치	black·블랙·BLACK	색상·사이즈 별칭을 표준값으로 정규화
관계 오류	같은 그룹에 다른 카테고리	product_group_id 단위로 그룹 전체 비교
가격 관계 오류	sale_price 가 price 보다 큼	형식·0 이하·정상가 초과를 분리 검수
공급사 구조 차이	vendor_sku / style_id	JSON 프로필로 표준 컬럼에 매핑
적재 추적 부족	표준 CSV 가 파일로만 남음	PostgreSQL staging 배치 적재·조회

사용자 흐름

사용자 흐름



사용자가 받는 결과

검수 상태, 오류 항목, 상품 ID, 오류 이유, 수정 권장사항, 위험 수준을 한글로 제공하고 필터 결과를 CSV 로 내려받을 수 있습니다. ETL 적재 이력은 별도 API 에서 배치와 상품을 조회합니다.

MVP 범위

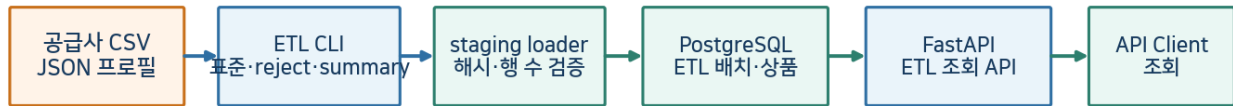
규칙 기반 검수와 공급사 ETL·staging 적재·조회는 구현하였지만, ETL 실행 웹 API 와 운영 상품 upsert 는 아직 지원하지 않습니다. AI 모델을 핵심 기능처럼 과장하지 않고 데이터 품질 검수·백엔드·DB 설계 경험을 중심으로 설명합니다.

2. 시스템 구조와 책임 분리

화면, API, 작업 큐, 검수 저장, ETL staging 의 역할을 나누어 같은 기능이 중복되지 않도록 하였습니다.

CatalogGuard Lite 전체 구조

검수 흐름과 ETL staging 적재·조회 흐름을 분리하고 PostgreSQL에서 이력을 관리합니다.



CLI 기반 변환·적재 / 읽기 전용 조회 API



동기 요청: FastAPI → PostgreSQL

비동기 요청: FastAPI → Redis → Celery → PostgreSQL

조회 응답은 FastAPI를 통해 화면으로 반환 · 운영 Redis·Celery 배포는 미검증

계층별 역할

구성	주요 책임	설계 판단
Streamlit	파일 사전 검증, 마스킹 미리보기, 결과·통계·다운로드	전체 검수 규칙을 다시 실행하지 않음
FastAPI 검수	동기·비동기 요청, 검수 단일 기준, 검수 이력 API	화면과 저장 결과의 기준을 서버로 통일
Redis·Celery	작업 상태와 백그라운드 검수	긴 검수를 HTTP 요청 수명과 분리
ETL CLI	공급사 변환, 정상·reject·summary, staging 적재	공급사별 코드를 만들지 않고 프로필 재사용
FastAPI ETL 조회	배치 목록·상세와 staging 상품 페이지	읽기 전용 조회 계층으로 분리
PostgreSQL	검수 이력과 ETL 배치·상품, 중복·제약조건	브라우저 재시작·동시 요청에도 데이터 유지

핵심 기술 판단

단일 검수 기준 — 서버 검수 결과를 Streamlit 화면·통계·필터·다운로드에 재사용하여 이중 검수를 제거하였습니다.

ETL 조회 분리 — staging 조회 함수는 commit·rollback을 실행하지 않고 SQL 페이지네이션과 count 만 담당합니다.

범위 분리 — ETL 조회 API는 검수를 자동 실행하지 않으며 표준 CSV 검수는 별도 요청으로 수행합니다.

3. 핵심 검수 기능

등록된 검수 규칙을 공통 ValidationIssue 형태로 통일하여 API 와 화면에 같은 결과를 전달합니다.

검수 범위

분류	검수 내용	대표 예시
입력·형식	필수값, 카테고리, 재고, price·sale_price 형식	price="가격문의" → 가격 오류
중복	상품 ID, 상품명 후보, 완전 중복	같은 속성이 모두 같은 상품
패션 옵션	색상·사이즈 비표준, 옵션 조합 중복	블랙 → BLACK, medium → M
그룹 관계	같은 그룹의 카테고리 불일치	TOP 과 BOTTOM 이 같은 그룹에 존재
가격	0 이하, 정상가·할인가 관계, 카테고리 이상치	price 50,000 / sale_price 60,000
내용·보안	상품명-카테고리, 금지어, 개인정보 의심	설명에 이메일·전화번호 포함

패션 데이터 정규화 예시

BLACK black / 블랙 / BLACK	M medium / M	XXL 2XL / XXL	FREE 프리사이즈 / FREE
------------------------------------	------------------------	-------------------------	-----------------------------

그룹 단위 판단

개별 행만 검사하면 같은 상품 그룹 안에서 발생하는 관계 오류를 찾기 어렵습니다. 다수결로 임의의 정답 카테고리를 만들지 않고 불일치가 존재한다는 사실을 참여 상품 전체에 연결하였으며, 결과는 원본 행 순서를 유지합니다.

결과 계약

모든 규칙은 severity, product_id, product_group_id, message 를 가진 ValidationIssue 로 통일합니다. 화면에서는 검수 상태·오류 항목·오류 이유·수정 권장사항·위험 수준으로 변환합니다.

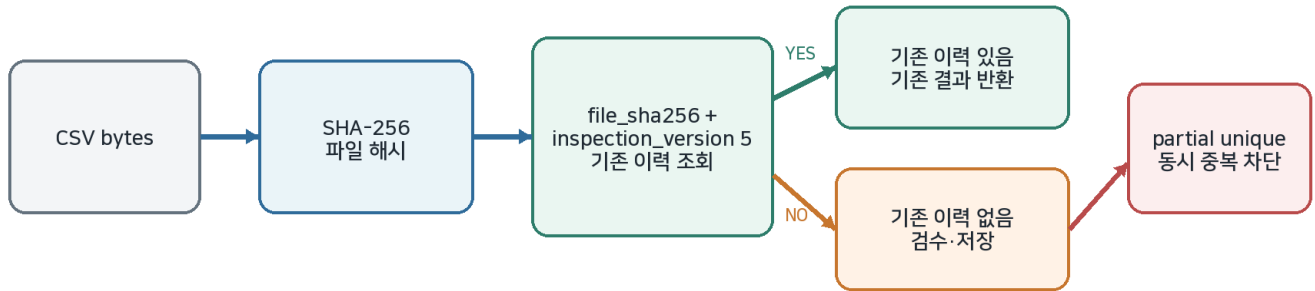
안전한 결과 표시

내부 상세 값은 JSON 구조로 저장해 작은따옴표와 쉼표가 포함된 값도 안전하게 전달합니다.
손상되거나 예상하지 못한 JSON 은 사용자 화면 전체를 중단시키지 않고 기본 안내 문구로 처리합니다.
다운로드 CSV 는 Excel 수식 삽입 위험이 있는 =, +, -, @ 시작 값을 복사본에서만 안전하게 처리합니다.

4. 동일 CSV 결과 재사용과 검수 버전 관리

불필요한 재검수를 줄이되 규칙 변경 후에는 같은 파일도 새 기준으로 다시 검사하도록 설계했습니다.

동일 CSV 결과 재사용과 동시 요청 방어



문제와 해결

문제	해결 방법	효과
같은 파일 반복 업로드	CSV bytes 의 SHA-256 해시 생성	파일명과 무관하게 같은 내용 식별
새 규칙인데 과거 결과 재사용	file_sha256 + inspection_version 조회	버전이 바뀌면 같은 CSV 도 재검수
동시에 같은 요청이 들어옴	PostgreSQL partial unique index	애플리케이션 조회 이후 경쟁 요청도 DB 가 차단
화면 세션 내 반복 제출	saved hash 와 활성 job_id 확인	불필요한 POST·GET 호출 감소

inspection_version 5 로 올린 이유

sale_price 형식·0 이하 값·정상가보다 큰 관계를 검사하는 규칙이 추가되었습니다. 파일 해시만 사용하거나 버전 4 를 유지하면 과거 결과가 재사용되어 새로운 할인 가격 검수가 실행되지 않을 수 있으므로 버전을 5 로 변경하였습니다.

DB migration 이 없었던 이유

sale_price 원본 상품 전체를 검수 이력 테이블에 저장하지 않고 검수 실행과 결과만 저장하므로 기존 검수 DB 컬럼 구조는 바뀌지 않았습니다. ETL staging 테이블은 이후 별도 migration 으로 추가하였습니다.

면접에서 설명할 핵심

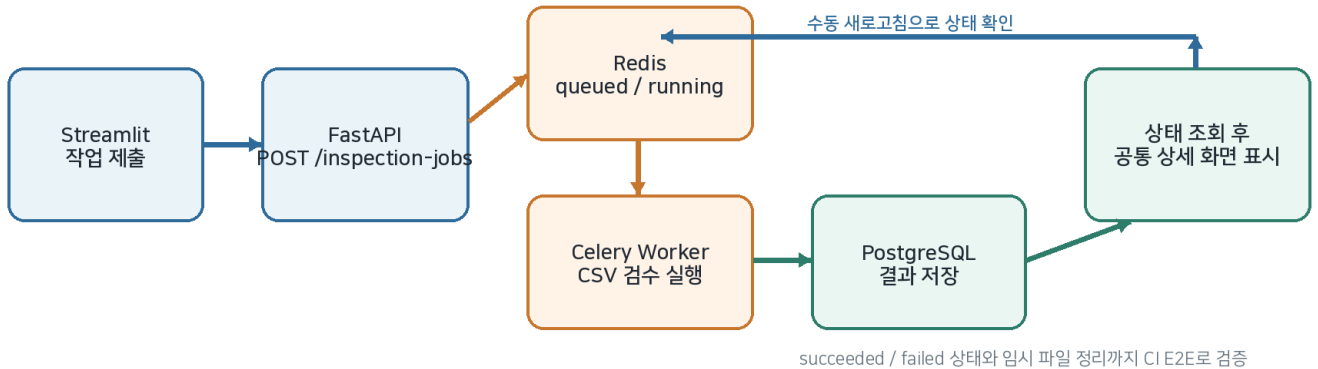
중복 저장 방지와 불필요한 검수 실행 감소는 다른 문제이므로 API 사전 조회와 DB 제약을 함께 사용했습니다.

검수 버전은 기능 소개용 숫자가 아니라 결과 재사용의 정확성을 보장하는 식별자입니다.

5. 비동기 처리와 자동 검증

긴 검수 작업을 요청 수명에서 분리하고 실제 서비스들을 연결한 흐름을 GitHub Actions 에서 확인했습니다.

Redis·Celery 비동기 검수 흐름



비동기 기능

항목	구현 내용
작업 상태	queued, running, succeeded, failed
제출·조회	작업 제출 API와 명시적 상태 조회 API
결과 표시	완료 후 동기 검수와 같은 상세 결과 렌더러 사용
정리	성공·실패 후 임시 CSV 파일 정리
중복 방지	같은 세션의 활성화 job_id와 파일 해시로 반복 제출 방지

GitHub Actions Test workflow

순서	자동 검증 범위
1	PostgreSQL 18·Redis 7.4 서비스 컨테이너 시작
2	Alembic upgrade head와 E2E 제외 전체 pytest 실행
3	Celery Worker와 FastAPI 프로세스 시작
4	/health·/ready, 비동기 제출, polling, 결과 저장, 동일 파일 재사용, 임시 파일 정리
5	Streamlit 시작 후 /_stcore/health HTTP 200·ok 확인

배포·운영 검증 범위

Railway — FastAPI /health·/ready와 구조화 요청 로그를 확인하였습니다.

AWS — EC2·RDS staging에서 Docker, Alembic, TLS DB 연결, FastAPI와 별도 Streamlit 흐름을 검증한 뒤 비용 절감을 위해 리소스를 중지하였습니다.

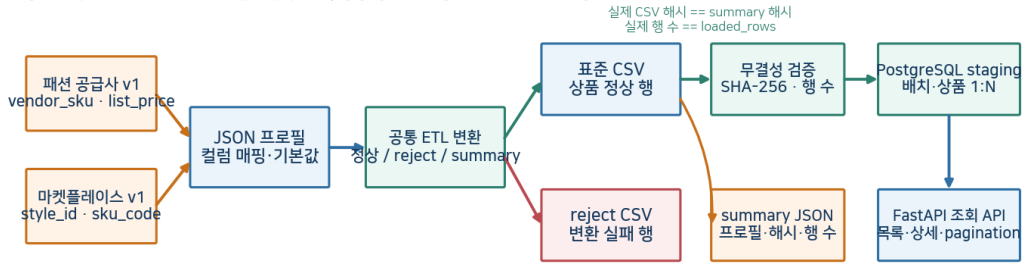
범위 제한 — 운영 환경 Redis·Celery 배포는 아직 검증하지 않았으며 현재 비동기 근거는 로컬·CI E2E입니다.

6. 공급사 ETL 과 PostgreSQL staging 적재

공급사별 구조 차이를 프로파일로 흡수하고, 변환된 정상 상품을 검증 후 임시 적재 영역에 배치 단위로 저장했습니다.

공급사 ETL → PostgreSQL staging → 조회 API

공급사 구조 차이는 JSON 프로파일로 흡수하고, 정상 상품은 검증 후 배치 단위로 저장합니다.



동일 input hash + profile + version은 기존 배치 재사용 · 저장 실패 시 전체 rollback

합성 공급사 프로파일 비교

프로파일	원본 → 표준 컬럼	설계 의미
패션 공급사 v1	vendor_sku → product_group_id, product_id	그룹 ID 와 개별 ID 를 한 키로 처리
패션 공급사 v1	discount_price → sale_price	선택 할인 가격 컬럼 처리
마켓플레이스 v1	style_id → product_group_id	상품 그룹 키를 별도로 유지
마켓플레이스 v1	sku_code → product_id	같은 그룹 아래 개별 SKU 유지
마켓플레이스 v1	promo_price → sale_price	할인 가격 관계 검수로 전달

staging DB 설계와 안전장치

구성	역할	안전장치
etl_load_runs	원본 파일·프로파일·해시·적재 행 수를 배치 단위로 저장	input hash + profile + version unique
catalog_products_staging	정상 상품 행을 배치 ID 와 연결하여 저장	stock·price·sale_price CHECK
1:N 관계	배치 1 개에 상품 여러 개 연결	Foreign Key 와 DB cascade
트랜잭션	배치 생성과 상품 저장을 하나로 처리	일부 실패 시 전체 rollback

무결성 검증

실제 표준 CSV 의 SHA-256 과 summary JSON 의 output_file_sha256, 실제 행 수와 loaded_rows 가 모두 일치할 때만 적재합니다. 운영 DB 가 아닌 임시 PostgreSQL 18.4 에서 migration·중복·rollback·constraint·cascade 를 검증하였습니다.

7. ETL 적재 배치 조회 API

PostgreSQL staging 에 저장된 배치와 상품을 외부에서 확인할 수 있도록 읽기 전용 FastAPI 조회 계층을 추가했습니다.

API 계약

엔드포인트	주요 파라미터	응답과 처리
GET /api/v1/etl-loads	limit, offset, filename, profile_name	배치 목록·total, 최신 적재 우선 정렬
GET /api/v1/etl-loads/{id}	product_limit, product_offset	배치 메타데이터·SHA-256·상품 페이지, 없는 배치 404

검색·정렬·페이지네이션

설계 항목	구현 판단	검증 내용
파일명·프로필 검색	앞뒤 공백 제거, 공백 검색어 무시, AND 조건	대소문자 구분 없는 부분 검색
LIKE 특수문자	%, _, W를 wildcard 가 아닌 실제 문자로 escape	실제 문자 포함 fixture 만 반환
배치 정렬	created_at DESC, 같은 시각은 id DESC	최신 배치가 먼저 표시
상품 정렬	catalog_products_staging.id ASC	원본 적재 순서에 가까운 안정적 결과
페이지네이션	Python 자르기 대신 SQL LIMIT·OFFSET	전체 relationship 로딩 방지
목록 total	items 와 count 에 동일 필터 함수 적용	페이지 수 불일치 방지

책임 분리와 N+1 방지

라우터는 Query·Path 검증, 404, Pydantic 응답 변환만 담당하고 SQL 은 db/etl_query_service.py 로 분리하였습니다.

배치 정보·상품 count·상품 페이지를 별도 SELECT 로 조회하여 relationship 전체 lazy loading 과 N+1 을 피했습니다.

목록에서는 긴 SHA-256 과 상품 전체를 제외하고 상세 요청에서만 제공하여 응답 크기를 줄였습니다.

TDD 와 테스트 수정 사례

신규 테스트를 먼저 작성해 ImportError RED 를 확인한 뒤 구현하였습니다. PostgreSQL 검증 중 underscore 검색 기대값이 fixture 와 맞지 않는 테스트 오류를 발견했고, 구현을 바꾸지 않고 실제 _ 문자를 포함한 배치 3 건을 기대하도록 테스트를 바로잡았습니다.

8. 검증 결과, 기술 판단과 한계

숫자를 과장하지 않고 어떤 환경에서 무엇을 검증했는지와 아직 지원하지 않는 범위를 함께 기록했습니다.

검증 결과

검증 항목	결과	해석
PostgreSQL 전체 pytest	894 passed, 2 deselected, 3 warnings	임시 PostgreSQL 18.4 에서 DB 테스트 포함, 실패 ·skip 0
GitHub Actions	Test #35 success	전체 894 passed, 비동기 E2E 1 passed, Streamlit smoke success
ETL staging	migration·적재·중복·rollback·constraint·cascade	임시 DB·포트·산출물 정리 완료
ETL 조회 API	검색·특수문자·정렬·pagination·404·NULL	목록/total 동일 조건과 배치별 상품 격리
Streamlit 서버	/_score/health 200, ok	서버 시작 스모크이며 브라우저 전체 기능 검증은 아님

동기 검수 벤치마크

행 수	1 회 중앙값	연속 2 회	Python peak
100	0.009544 초	0.018815 초	0.177 MiB
1,000	0.074266 초	0.150817 초	1.603 MiB
5,000	0.371740 초	0.867245 초	6.485 MiB
10,000	0.875495 초	1.645721 초	12.939 MiB

개발 PC 합성 데이터 측정이며 운영 트래픽 성능을 보증하지 않습니다. 같은 검수를 두 번 실행했을 때의 비용을 근거로 Streamlit·FastAPI 이중 검수 제거를 우선했습니다.

현재 한계

구분	현재 상태
운영 비동기	Redis·Celery 운영 배포 미검증
ETL 실행	CLI 만 지원, 실행 웹 API 와 Streamlit ETL 이력 화면 미구현
데이터 유입	합성 공급사 프로필 2 종, 수동 선택, CSV 만 지원
상품 반영	운영 상품 upsert·변경 이력·reject DB 저장 미지원
대용량 처리	streaming·중분 ETL·동시 트래픽 측정 미지원

30 초 소개

면접 답변

CatalogGuard Lite 는 서로 다른 합성 공급사 CSV 를 JSON 프로필로 표준화하고 누락·중복·가격·카테고리·개인정보 문제를 검수하는 데이터 품질 서비스입니다. FastAPI 와 PostgreSQL 로 검수 이력과 ETL staging 배치를 관리하고, 동일 CSV 는 파일 해시와 검수 버전으로 기존 결과를 재사용하도록 설계했습니다. ETL 배치·상품 조회 API 와 Redis·Celery 비동기 흐름을 PostgreSQL 통합 테스트와 GitHub Actions E2E 로 검증했습니다.

최신 기준: commit c15db41 · inspection_version 5 · GitHub Actions Test #35 success